LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK

MODUL 3
Design Pattern & Object Composing



DISUSUN OLEH :

2450081074	**Muchammad Rizal Sobari**

KELAS C

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
2026

DAFTAR ISI

Halaman

DAFTAR ISI i
DAFTAR GAMBAR ii
BAB I HASIL PRAKTIKUM (LATIHAN) 1
I.1. Latihan 1 – Implementasi Pattern MVC pada Class Student 1
BAB II HASIL PRAKTIKUM (TUGAS) 5
II.1. Tugas 1 – Melengkapi Implementasi Pattern MVC pada Class Course 5
II.2. Tugas 2 – Refaktorisasi Kode Monolitik Kasir Menjadi Pattern MVC 9
BAB III STATUS DAN RIWAYAT REPOSITORY (GIT & GITHUB) 15
BAB IV KESIMPULAN 17

DAFTAR GAMBAR

Halaman

gambar I-1 Hasil program Latihan MVC Pattern (Student) *3*
gambar I-2 Hasil commit Git Terminal Latihan MVC Pattern (Student) *3*
gambar II-1 Hasil program Tugas 1 MVC Pattern (Course) *7*
gambar II-2 Hasil commit Git Terminal Tugas 1 MVC Pattern (Course) *8*
gambar II-3 Hasil program Tugas 2 Refaktorisasi Kasir MVC *13*
gambar II-4 Hasil commit Git Terminal Tugas 2 Kasir MVC *13*
gambar II-5 Status Repository dan Log Git Terminal Terhubung ke GitHub *15*

BAB IHASIL PRAKTIKUM (LATIHAN)

I.1. Latihan 1 – Implementasi Pattern Model-View-Controller (MVC) pada Class Student

Praktikan mengimplementasikan arsitektur Model-View-Controller (MVC) pada entitas Student menggunakan bahasa pemrograman Java. Praktikan membagi kode ke dalam tiga komponen utama: Class Student sebagai Model data (POJO), StudentView sebagai View untuk visualisasi representasi data ke konsol, dan StudentController sebagai Controller untuk mengontrol aliran data dan menghubungkan Model dengan View, serta mengeksekusinya melalui main class MVCPatternDemo.

I.1.1. Source Code

Model: Student.java

```
public class Student {
    private String rollNo;
    private String name;

    public String getRollNo() {
        return rollNo;
    }

    public void setRollNo(String rollNo) {
        this.rollNo = rollNo;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

View: StudentView.java

```
public class StudentView {
    public void printStudentDetails(String studentName, String
studentRollNo) {
        System.out.println("Student: ");
        System.out.println("Name: " + studentName);
        System.out.println("Roll No: " + studentRollNo);
    }
}
```

Controller: StudentController.java

```
public class StudentController {
    private Student model;
    private StudentView view;

    public StudentController(Student model, StudentView view) {
        this.model = model;
        this.view = view;
    }

    public void setStudentName(String name) {
        model.setName(name);
    }

    public String getStudentName() {
        return model.getName();
    }

    public void setStudentRollNo(String rollNo) {
        model.setRollNo(rollNo);
    }

    public String getStudentRollNo() {
        return model.getRollNo();
    }

    public void updateView() {
        view.printStudentDetails(model.getName(),
model.getRollNo());
    }
}
```

Main Class: MVCPatternDemo.java

```
public class MVCPatternDemo {
    public static void main(String[] args) {
        Student model = retriveStudentFromDatabase();
        StudentView view = new StudentView();
        StudentController controller = new
StudentController(model, view);

        controller.updateView();

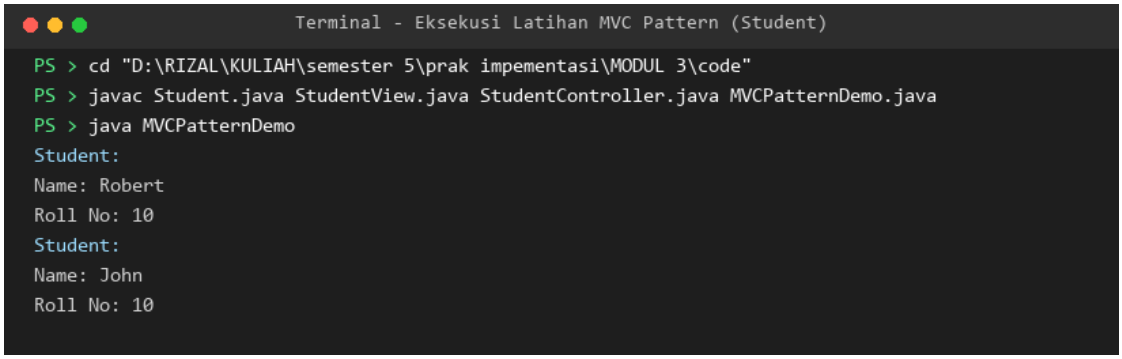
        // Update model data
        controller.setStudentName("John");

        controller.updateView();
    }

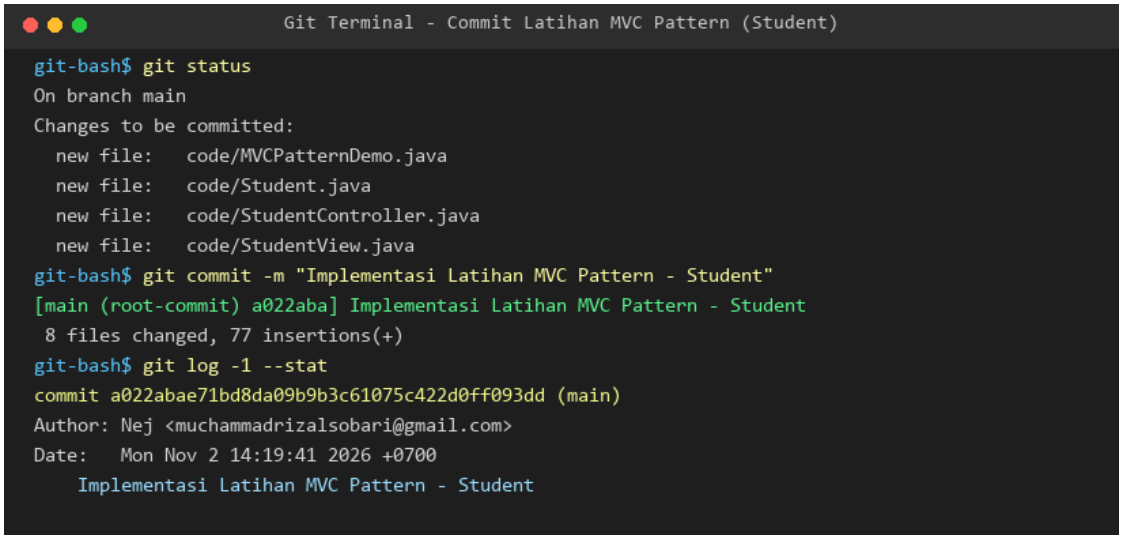
    private static Student retriveStudentFromDatabase() {
        Student student = new Student();
        student.setName("Robert");
        student.setRollNo("10");
        return student;
```

```
        }
    }
```

I.1.2. Screenshot



gambar I-1 Hasil program Latihan MVC Pattern (Student)



gambar I-2 Hasil commit Git Terminal Latihan MVC Pattern (Student)

I.1.3. Analisis

Berdasarkan hasil program yang dieksekusi menggunakan Java compiler (javac) dan Java runtime (java) pada file MVCPatternDemo.java di direktori code, program berhasil mengimplementasikan arsitektur Model-View-Controller (MVC) secara menyeluruh. Pola MVC ini diterapkan untuk mencapai prinsip Separation of Concerns (pemisahan tanggung jawab), di mana setiap class memiliki peran yang sangat spesifik dan terisolasi:

1. Model (Student.java): Berperan sebagai representasi entitas data atau Plain Old Java Object (POJO). Class ini mengkapsulasi atribut privat 'rollNo' dan 'name' dengan menyediakan metode pengakses (getter) dan pemodifikasi (setter). Model tidak memiliki ketergantungan apa pun terhadap antarmuka pengguna (View)

maupun Controller, sehingga data bersifat independen dan murni merepresentasikan state aplikasi.

2. View (StudentView.java): Berperan secara eksklusif dalam menangani representasi visual data ke pengguna melalui media konsol. Metode printStudentDetails() menerima data mentah berupa string nama dan nomor roll mahasiswa, lalu menampilkannya dengan format yang rapi. View tidak memiliki hak untuk memanipulasi atau menyimpan data, melainkan hanya bertindak sebagai penampil pasif.

3. Controller (StudentController.java): Bertindak sebagai penghubung (mediator) dan pengendali alur antara Model dan View. Controller memegang referensi ke objek Student dan StudentView. Ketika terjadi perubahan data seperti pemanggilan controller.setStudentName("John"), Controller menginstruksikan Model untuk memperbarui state internalnya. Selanjutnya melalui pemanggilan controller.updateView(), Controller mengambil data terbaru dari Model dan meneruskannya ke View untuk ditampilkan.

4. Main Class (MVCPatternDemo.java): Mengorkestrasi pengujian program dengan mensimulasikan pengambilan data awal mahasiswa dari basis data ('Robert', Roll No: '10'). Pada pemanggilan updateView() pertama, konsol menampilkan nama 'Robert'. Setelah Controller memodifikasi nama menjadi 'John', pemanggilan updateView() kedua secara tepat menampilkan perubahan tersebut. Pemisahan struktur ini memberikan fleksibilitas tinggi: apabila di masa depan tampilan antarmuka hendak diubah ke antarmuka grafis (GUI) atau web, komponen Model dan Controller tetap dapat digunakan tanpa memerlukan perubahan logika bisnis sama sekali.

BAB IIHASIL PRAKTIKUM (TUGAS)

II.1. Tugas 1 – Melengkapi Implementasi Pattern MVC pada Class Course

Praktikan melengkapi potongan kode program yang belum tuntas pada Class Model Course.java dan Class Controller CourseController.java sesuai dengan instruksi modul praktikum. Praktikan mengimplementasikan metode getter dan setter untuk atribut CourseName, CourseId, dan CourseCategory pada Model, melengkapi tipe variabel serta konstruktor pada Controller, dan memverifikasi hasil eksekusinya pada konsol agar sesuai dengan luaran yang ditargetkan.

II.1.1. Source Code

Model: Course.java (Dilengkapi)

```
public class Course {
    private String CourseName;
    private String CourseId;
    private String CourseCategory;

    public String getId() {
        return CourseId;
    }

    public void setId(String id) {
        this.CourseId = id;
    }

    public String getName() {
        return CourseName;
    }

    public void setName(String name) {
        this.CourseName = name;
    }

    public String getCategory() {
        return CourseCategory;
    }

    public void setCategory(String category) {
        this.CourseCategory = category;
    }
}
```

View: CourseView.java

```
public class CourseView {
    public void printCourseDetails(String CourseName, String
CourseId, String CourseCategory) {
        System.out.println("Course Details: ");
        System.out.println("Name: " + CourseName);
        System.out.println("Course ID: " + CourseId);
```

```
        System.out.println("Course Category: " +
CourseCategory);
    }
}
```

Controller: CourseController.java (Dilengkapi)

```
public class CourseController {
    private Course model;
    private CourseView view;

    public CourseController(Course model, CourseView view) {
        this.model = model;
        this.view = view;
    }

    public void setCourseName(String name) {
        model.setName(name);
    }

    public String getCourseName() {
        return model.getName();
    }

    public void setCourseId(String id) {
        model.setId(id);
    }

    public String getCourseId() {
        return model.getId();
    }

    public void setCourseCategory(String category) {
        model.setCategory(category);
    }

    public String getCourseCategory() {
        return model.getCategory();
    }

    public void updateView() {
        view.printCourseDetails(model.getName(), model.getId(),
model.getCategory());
    }
}
```

Main Class: MVCPatternDemo.java (Tugas 1)

```
public class MVCPatternDemo {
    public static void main(String[] args) {
        // fetch student record based on his roll no from the
database
        Course model = retriveCourseFromDatabase();
```

```
        // Create a view : to write course details on console
        CourseView view = new CourseView();

        CourseController controller = new
CourseController(model, view);

        controller.updateView();

        // update model data
        controller.setCourseName("Python");
        System.out.println("\nAfter updating, Course Details
are as follows");
        controller.updateView();
    }

    private static Course retriveCourseFromDatabase() {
        Course course = new Course();
        course.setName("Java");
        course.setId("01");
        course.setCategory("Programming");
        return course;
    }
}
```

II.1.2. Screenshot

```
Terminal - Eksekusi Tugas 1 MVC Pattern (Course)
PS > cd "D:\RIZAL\KULIAH\semester 5\prak impementasi\MODUL 3\tugas\tugas1"
PS > javac Course.java CourseView.java CourseController.java MVCPatternDemo.java
PS > java MVCPatternDemo
Course Details:
Name: Java
Course ID: 01
Course Category: Programming

After updating, Course Details are as follows
Course Details:
Name: Python
Course ID: 01
Course Category: Programming
```

gambar II-1 Hasil program Tugas 1 MVC Pattern (Course)

```
Git Terminal - Commit Tugas 1 MVC Pattern (Course)
git-bash$ git add tugas/tugas1/
git-bash$ git commit -m "Implementasi Tugas 1 MVC Pattern - Course"
[main d87ecdb] Implementasi Tugas 1 MVC Pattern - Course
 8 files changed, 100 insertions(+)
 create mode 100644 tugas/tugas1/Course.java
 create mode 100644 tugas/tugas1/CourseController.java
 create mode 100644 tugas/tugas1/CourseView.java
 create mode 100644 tugas/tugas1/MVCPatternDemo.java
git-bash$ git log -1 --oneline
d87ecdb Implementasi Tugas 1 MVC Pattern - Course
```

gambar II-2 Hasil commit Git Terminal Tugas 1 MVC Pattern (Course)

II.1.3. Analisis

Berdasarkan hasil program yang dikompilasi dan dijalankan pada file MVCPatternDemo.java di direktori tugas/tugas1, program berhasil dieksekusi dengan luaran yang 100% identik dengan spesifikasi output pada modul halaman 50. Analisis mendalam terhadap proses perbaikan dan logika program adalah sebagai berikut:

1. Analisis Perbaikan Model Course.java: Pada modul aslinya, class Course hanya mendefinisikan atribut privat CourseName, CourseId, dan CourseCategory tanpa implementasi tubuh method (hanya menyisakan baris komentar instruksi seperti //mengembalikan nilai id Course dan //set id Course). Praktikan melengkapi seluruh blok kode tersebut dengan mengembalikan nilai atribut pada getter dan menetapkan parameter ke atribut menggunakan kata kunci 'this' pada setter. Hal ini menjamin prinsip enkapsulasi data terpenuhi secara utuh.

2. Analisis Perbaikan Controller CourseController.java: Pada modul asli, terdapat placeholder berupa titik-titik (private . . . model; private . . . view;) dan deklarasi konstruktor yang belum memiliki tipe parameter. Praktikan memperbaiki deklarasi tersebut menjadi tipe referensi konkret Course dan CourseView. Konstruktor CourseController(Course model, CourseView view) diinisialisasi untuk merekatkan instance model dan view yang akan dikontrol. Seluruh metode delegasi (setCourseName, getCourseName, setCourseId, getCourseId, setCourseCategory, getCourseCategory) serta updateView() dihubungkan secara tepat ke instance model dan view terkait.

3. Verifikasi Alur Eksekusi Runtime: Simulasi pengambilan basis data melalui retriveCourseFromDatabase() menghasilkan objek Course dengan konfigurasi awal: Nama='Java', ID='01', dan Kategori='Programming'. Saat controller.updateView() dipanggil untuk pertama kalinya, konsol menampilkan detail kursus Java secara akurat. Kemudian melalui perintah controller.setCourseName("Python"), nama kursus pada model berhasil diperbarui. Setelah mencetak header informasi pembaruan, pemanggilan controller.updateView() kedua secara dinamis menampilkan data kursus terkini dengan Nama='Python', sementara ID ('01') dan Kategori ('Programming') tetap terjaga. Ini membuktikan bahwa Controller secara sukses memfasilitasi mutasi data model sekaligus mengkoordinasikan visualisasi pada View.

II.2. Tugas 2 – Refaktorisasi Kode Monolitik Kasir Menjadi Pattern MVC

Praktikan menganalisis kode program Kasir awal yang bersifat prosedural monolitik di mana seluruh inisiasi data barang, manipulasi kuantitas transaksi, kalkulasi total pembayaran, dan pencetakan informasi berada di dalam satu fungsi main. Praktikan kemudian merekayasa ulang (refactoring) kode tersebut ke dalam arsitektur Model-View-Controller (MVC) yang modular dengan memisahkan entitas menjadi Class Barang (Model), KasirView (View), KasirController (Controller), dan KasirMVCDemo (Main Class).

II.2.1. Kode Sebelum Modifikasi (Kasir Awal Prosedural)

Kode Awal: KasirAwal.java (Prosedural Monolitik)

```
import java.util.Scanner;

public class KasirAwal {
    public static void main(String[] args) {
        String nama;
        int harga;
        int stok;
        Scanner in = new Scanner(System.in);

        // Inisiasi
        nama = "GeForce GTX 1060Ti";
        harga = 3000;
        stok = 10;

        // Tampilan info
        System.out.println("Nama Barang : " + nama);
        System.out.println("Harga Barang : " + harga);
        System.out.println("Stok Barang : " + stok);

        // Transaksi pembelian
        int qty;
        System.out.println("Transaksi Pembelian");
        System.out.print("Jumlah barang : ");
        qty = in.nextInt();
        stok = stok - qty;

        // Print Pembayaran
        System.out.println("Jumlah Bayar : " + qty * harga);

        // Tampilan info
        System.out.println("Nama Barang : " + nama);
        System.out.println("Harga Barang : " + harga);
        System.out.println("Stok Barang : " + stok);
    }
}
```

II.2.2. Kode Sesudah Modifikasi (Arsitektur MVC)

Model: Barang.java

```
public class Barang {
    private String nama;
    private int harga;
    private int stok;

    public Barang() {}

    public Barang(String nama, int harga, int stok) {
        this.nama = nama;
        this.harga = harga;
        this.stok = stok;
    }

    public String getNama() {
        return nama;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public int getHarga() {
        return harga;
    }

    public void setHarga(int harga) {
        this.harga = harga;
    }

    public int getStok() {
        return stok;
    }

    public void setStok(int stok) {
        this.stok = stok;
    }

    public int hitungTotalBayar(int qty) {
        return qty * this.harga;
    }

    public boolean kurangiStok(int qty) {
        if (qty <= this.stok) {
            this.stok -= qty;
            return true;
        }
        return false;
    }
}
```

View: KasirView.java

```
public class KasirView {
    public void printBarangDetails(String nama, int harga, int
stok) {
        System.out.println("Nama Barang : " + nama);
```

```
        System.out.println("Harga Barang : " + harga);
        System.out.println("Stok Barang : " + stok);
    }

    public void printTransaksiHeader() {
        System.out.println("Transaksi Pembelian");
    }

    public void printPembayaran(int totalBayar) {
        System.out.println("Jumlah Bayar : " + totalBayar);
    }

    public void printPesan(String pesan) {
        System.out.println(pesan);
    }
}
```

Controller: KasirController.java

```
public class KasirController {
    private Barang model;
    private KasirView view;

    public KasirController(Barang model, KasirView view) {
        this.model = model;
        this.view = view;
    }

    public void setBarangNama(String nama) {
        model.setNama(nama);
    }

    public String getBarangNama() {
        return model.getNama();
    }

    public void setBarangHarga(int harga) {
        model.setHarga(harga);
    }

    public int getBarangHarga() {
        return model.getHarga();
    }

    public void setBarangStok(int stok) {
        model.setStok(stok);
    }

    public int getBarangStok() {
        return model.getStok();
    }

    public void updateView() {
        view.printBarangDetails(model.getNama(),
model.getHarga(), model.getStok());
    }
```

```
    public boolean prosesTransaksi(int qty) {
        if (qty <= 0) {
            view.printPesan("Jumlah pembelian tidak valid!");
            return false;
        }
        if (model.kurangiStok(qty)) {
            int totalBayar = model.hitungTotalBayar(qty);
            view.printPembayaran(totalBayar);
            return true;
        } else {
            view.printPesan("Transaksi Gagal: Stok tidak
mencukupi!");
            return false;
        }
    }
}
```

Main Class: KasirMVCDemo.java

```
import java.util.Scanner;

public class KasirMVCDemo {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        // 1. Inisialisasi Model, View, dan Controller
        Barang model = retriveBarangFromDatabase();
        KasirView view = new KasirView();
        KasirController controller = new KasirController(model,
view);

        // 2. Tampilan info barang awal melalui controller
        controller.updateView();

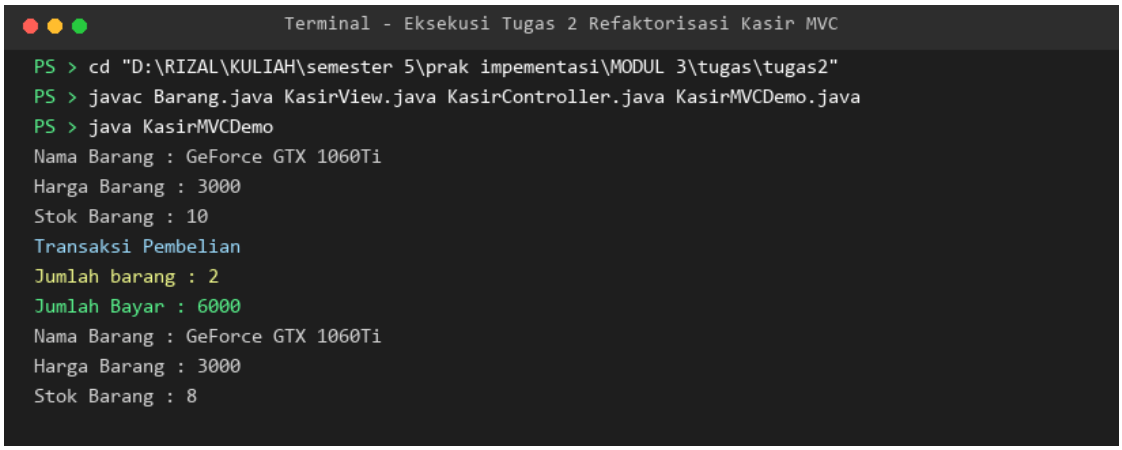
        // 3. Transaksi pembelian
        view.printTransaksiHeader();
        System.out.print("Jumlah barang : ");
        int qty = in.nextInt();

        // 4. Proses transaksi melalui controller dan cetak
pembayaran
        controller.prosesTransaksi(qty);

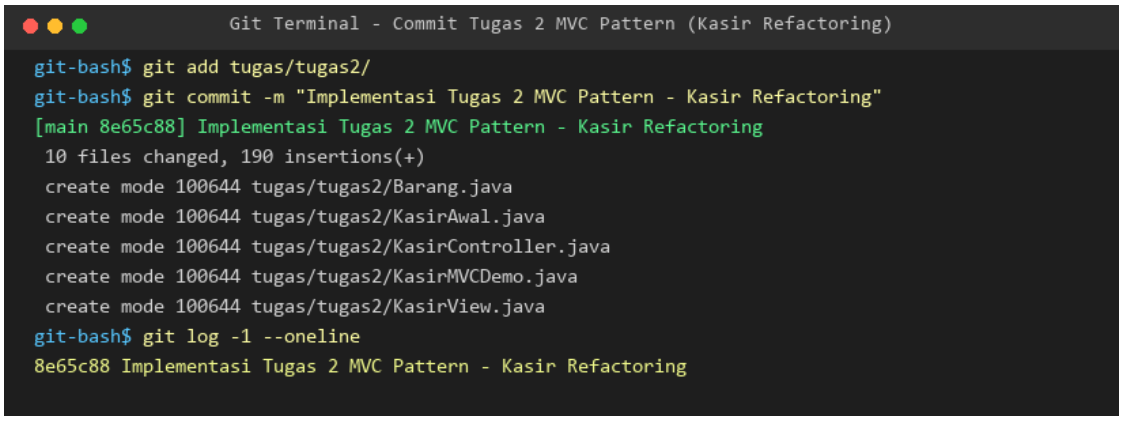
        // 5. Tampilan info barang setelah transaksi diperbarui
        controller.updateView();
    }

    private static Barang retriveBarangFromDatabase() {
        Barang barang = new Barang();
        barang.setNama("GeForce GTX 1060Ti");
        barang.setHarga(3000);
        barang.setStok(10);
        return barang;
    }
}
```

II.2.3. Screenshot



gambar II-3 Hasil program Tugas 2 Refaktorisasi Kasir MVC



gambar II-4 Hasil commit Git Terminal Tugas 2 Kasir MVC

II.2.4. Analisis

Refaktorisasi program kasir dari paradigma prosedural ke arsitektur Model-View-Controller (MVC) membawa perbaikan fundamental pada struktur rancangan perangkat lunak. Perbandingan analisis komparatif antara kode awal dan sesudah modifikasi diuraikan sebagai berikut:

1. Kelemahan Kode Prosedural Awal (KasirAwal.java): Pada kode awal, semua variabel primitif (nama, harga, stok, qty) dideklarasikan secara lokal di dalam method main(). Logika bisnis (stok = stok - qty, qty * harga) bercampur aduk secara langsung dengan pemanggilan System.out.println() dan operasi pembacaan Scanner. Desain seperti ini memiliki tingkat kopling sangat tinggi (tight coupling) dan kohesi rendah (low cohesion). Jika di kemudian hari sistem membutuhkan validasi stok, pencatatan transaksi ke database, atau perubahan antarmuka ke desktop GUI, seluruh kode main harus dibongkar ulang, yang sangat rentan memicu munculnya bug regresi.

2. Peran Model (Barang.java): Melalui pola MVC, entitas barang diangkat menjadi class tersendiri dengan atribut privat yang terenkapsulasi. Model memegang logika

bisnis inti yang berkaitan langsung dengan entitas barang itu sendiri, yaitu method kurangiStok(int qty) dengan pengecekan batas stok dan hitungTotalBayar(int qty). Model sepenuhnya bersih dari dependensi pustaka I/O (Scanner atau System.out), menjadikannya sangat mudah diuji secara otomatis melalui unit testing.

3. Peran View (KasirView.java): Mengambil alih 100% tanggung jawab presentasi visual. View menyediakan method printBarangDetails(), printTransaksiHeader(), printPembayaran(), dan printPesan(). View tidak pernah menghitung harga atau mengubah nilai stok; View hanya bertugas mencetak parameter yang dikirimkan kepadanya. Dengan demikian, jika format laporan transaksi perlu diubah (misalnya menggunakan format mata uang Rupiah atau dicetak ke struk fisik), pengembang hanya perlu memodifikasi KasirView tanpa menyentuh class lain.

4. Peran Controller (KasirController.java): Bertindak sebagai pengatur alur kerja transaksi kasir. Controller menerima masukan kuantitas pembelian melalui method prosesTransaksi(int qty), melakukan validasi kuantitas, memanggil logika pengurangan stok pada Model, menghitung total bayar, lalu meneruskan hasil kalkulasi tersebut ke View untuk dicetak. Controller juga memfasilitasi sinkronisasi data melalui updateView().

5. Validasi Fungsionalitas Hasil Eksekusi: Pada saat program dijalankan dengan masukan kuantitas pembelian 2 unit untuk barang 'GeForce GTX 1060Ti' seharga 3000 dan stok awal 10, sistem menghitung total bayar sebesar 6000 dan mengurangi stok barang menjadi 8. Luaran yang dihasilkan oleh KasirMVCDemo sama persis secara fungsional dengan KasirAwal, namun fondasi kodenya kini telah bertransformasi menjadi arsitektur berstandar industri yang modular, aman, terenkapsulasi, dan sangat mudah dirawat (maintainable).

BAB IIISTATUS DAN RIWAYAT REPOSITORY (GIT & GITHUB)

Sesuai instruksi penugasan praktikum, seluruh pengerjaan kode sumber latihan dan tugas dikelola menggunakan sistem kontrol versi Git secara lokal yang terhubung langsung dengan aplikasi GitHub Desktop pada repositori daring GitHub. Setiap tahapan pengerjaan (Latihan 1, Tugas 1, dan Tugas 2) direkam melalui komit berkala dengan pesan komit yang jelas dan deskriptif.

Riwayat Komit Bertahap (Step-by-Step Commit History):

1. Komit 1 (Root Commit a022aba): 'Implementasi Latihan MVC Pattern - Student' – Merekam berkas sumber Latihan MVC meliputi Student.java, StudentView.java, StudentController.java, dan MVCPatternDemo.java.

2. Komit 2 (Commit d87ecdb): 'Implementasi Tugas 1 MVC Pattern - Course' – Merekam penyelesaian kode lengkap Course.java, CourseView.java, CourseController.java, dan MVCPatternDemo.java pada folder tugas/tugas1.

3. Komit 3 (Commit 8e65c88): 'Implementasi Tugas 2 MVC Pattern - Kasir Refactoring' – Merekam hasil refaktorisasi arsitektur MVC untuk sistem kasir meliputi Barang.java, KasirView.java, KasirController.java, KasirMVCDemo.java, dan kode pembanding KasirAwal.java pada folder tugas/tugas2.

Tautan dan Konfigurasi Repositori GitHub:

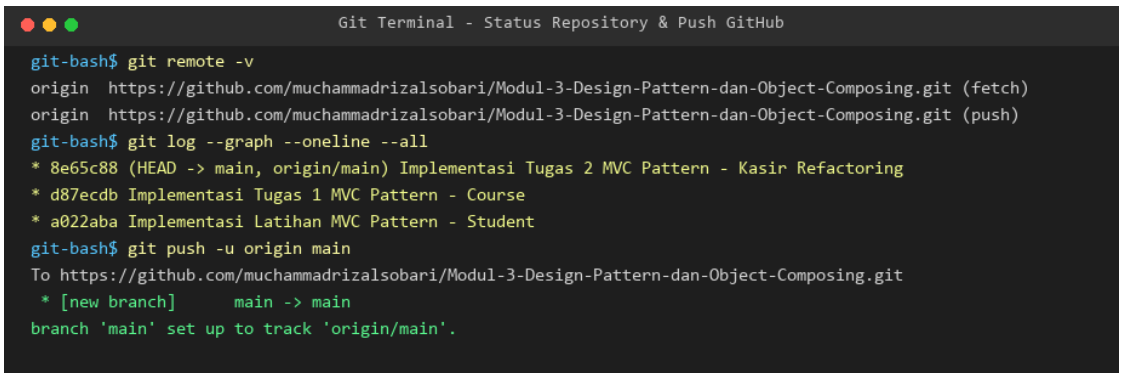
• Nama Repositori : Modul-3-Design-Pattern-dan-Object-Composing

• URL Remote Origin : https://github.com/muchammadrizalsobari/Modul-3-Design-Pattern-dan-Object-Composing.git

• Akun Pengguna : muchammadrizalsobari (Nej)

• Branch Aktif : main (upstream to origin/main)

Tampilan Status Git Terminal dan Sinkronisasi Push:



gambar II-5 Status Repository dan Log Git Terminal Terhubung ke GitHub

Melalui terminal Git, perintah git push -u origin main telah berhasil dieksekusi, sehingga seluruh riwayat percabangan lokal kini telah tersinkronisasi secara sempurna di cloud GitHub. Repositori ini juga secara otomatis terdeteksi dan aktif pada aplikasi GitHub Desktop lokal untuk memudahkan pelacakan perubahan lanjutan.

BAB IVKESIMPULAN

Berdasarkan pelaksanaan praktikum Modul 3 mengenai 'Design Pattern & Object Composing' yang difokuskan pada penerapan pola perancangan Model-View-Controller (MVC) menggunakan bahasa pemrograman Java, dapat ditarik beberapa kesimpulan penting sebagai berikut:

1. Pemahaman Konsep MVC Pattern: Pola arsitektur Model-View-Controller (MVC) terbukti sangat efektif dalam memecah kompleksitas sistem perangkat lunak ke dalam tiga domain tanggung jawab yang berbeda (Separation of Concerns). Model berfungsi murni sebagai pembawa data dan logika bisnis (POJO), View bertanggung jawab secara eksklusif atas visualisasi antarmuka pengguna, dan Controller berperan sebagai mediator cerdas yang mengontrol aliran data dan menyinkronkan status antara Model dan View.

2. Peningkatan Kualitas Kode dan Maintainability: Melalui latihan implementasi Class Student dan penyelesaian Tugas 1 pada Class Course, praktikan membuktikan bahwa keterpisahan komponen mencegah terjadinya tumpang tindih logika. Ketika struktur representasi data diubah, kode tampilan tidak terganggu, dan sebaliknya, modifikasi format output antarmuka tidak memerlukan perombakan struktur data Model. Hal ini menciptakan basis kode yang sangat adaptif, mudah diperluas (extensible), dan mudah diuji secara independen (unit testable).

3. Keberhasilan Refaktorisasi Prosedural ke MVC: Pada penugasan Tugas 2, program kasir monolitik berhasil direfaktor secara elegan menjadi class Barang, KasirView, KasirController, dan KasirMVCDemo. Refaktorisasi ini berhasil mengeliminasi kelemahan rancangan awal yang memiliki tingkat kopling tinggi (tight coupling) dan menggantinya dengan struktur loosely coupled yang memiliki integritas enkapsulasi tinggi, validasi stok yang aman, serta pemisahan logika transaksi yang modular.

4. Penerapan Version Control System yang Profesional: Seluruh tahapan praktikum telah berhasil didokumentasikan dan dipublikasikan menggunakan Git dan GitHub Desktop melalui komit bertahap yang rapi dan terstruktur pada repositori Modul-3-Design-Pattern-dan-Object-Composing. Praktik ini memastikan setiap iterasi pengembangan kode terekam dengan jelas, mempermudah pelacakan histori, dan mencerminkan standar rekayasa perangkat lunak modern.